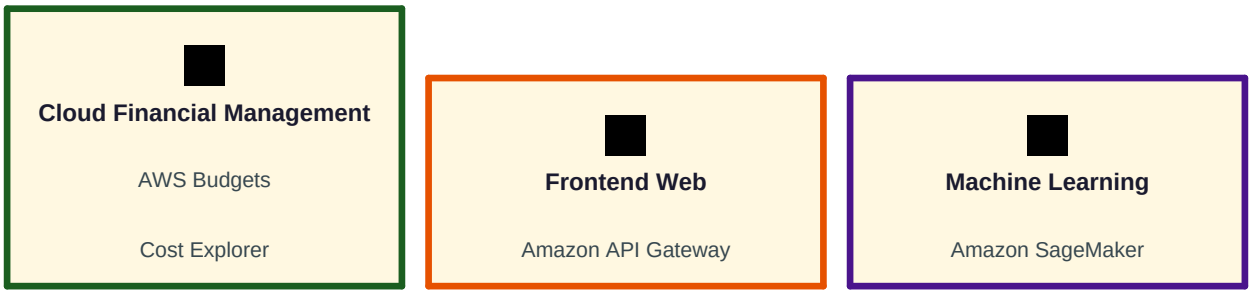
AWS Services Guide

Cloud Financial Management · Frontend Web · Machine Learning

AWS Budgets · Cost Explorer · Amazon API Gateway · Amazon SageMaker

Beginner-Friendly • In-Depth • Practical • Hands-On



AWS Solutions Architect Training Series • Version 1.0 • 2025

Table of Contents

■ Cloud Financial Management

◆ AWS Budgets

- ■ 1. Simple Introduction
- ■ 2. Core Concepts
- ■ 3. Architecture & Working
- ■ 4. Features
- ■ 5. Real-World Use Cases
- ■ 6. Practical Example
- ■ 7. Integrations
- ■ 8. Comparison

◆ AWS Cost Explorer

- ■ 1. Simple Introduction
- ■ 2. Core Concepts
- ■ 3. Architecture & Working
- ■ 4. Features
- ■ 5. Real-World Use Cases
- ■ 6. Practical Example
- ■ 7. Integrations
- ■ 8. Comparison

■ Frontend Web

◆ Amazon API Gateway

- ■ 1. Simple Introduction
- ■ 2. Core Concepts
- ■ 3. Architecture & Working
- ■ 4. Features
- ■ 5. Real-World Use Cases
- ■ 6. Practical Example
- ■ 7. Integrations
- ■ 8. Comparison

■ Machine Learning

◆ Amazon SageMaker

- ■ 1. Simple Introduction
- ■ 2. Core Concepts
- ■ 3. Architecture & Working
- ■ 4. Features
- ■ 5. Real-World Use Cases
- ■ 6. Practical Example
- ■ 7. Integrations
- ■ 8. Comparison

■ AWS Budgets

Set Custom Cost and Usage Thresholds — Get Alerted Before You Overspend

1. Simple Introduction

AWS Budgets is a **cost management service that lets you set custom budgets to track your AWS costs, usage, Savings Plans utilisation, and Reserved Instance coverage** — and automatically alerts you when you are approaching or have exceeded your defined thresholds. It is the proactive early-warning system for AWS spending.

■ **Real-World Analogy:** AWS Budgets is like setting a **spending limit on your credit card**. You tell the bank: 'Alert me when I reach 80% of my $5,000 limit, and block new charges if I hit 100%.' With AWS Budgets, you tell AWS: 'Alert my finance team when EC2 spend reaches $8,000 — and automatically shut down dev instances if we hit $10,000.' You stay in control without watching the bill every hour.

Business Problems Solved:

- Prevent bill shock — know about overspend before the invoice arrives
- Enforce cost guardrails across teams and projects without manual monitoring
- Track Reserved Instance and Savings Plans coverage to maximise discounts
- Automatically trigger cost-saving actions (stop EC2, deny IAM permissions) when thresholds are crossed
- Give each team, project, or cost centre its own budget with alerts

2. Core Concepts

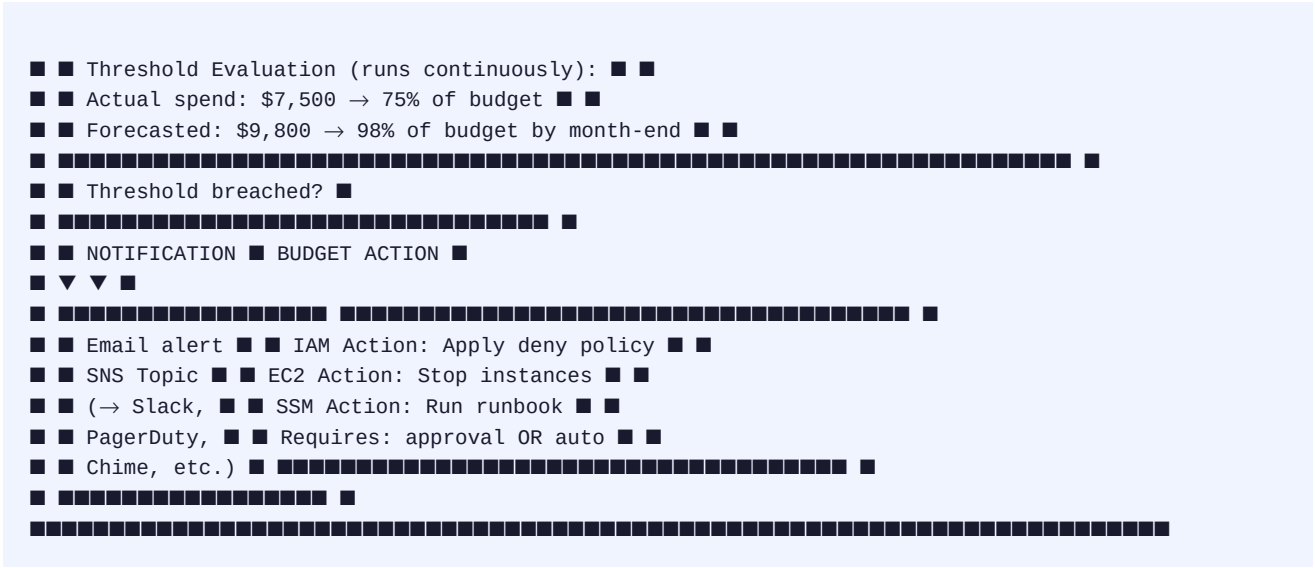
Term	Definition	Example
Budget	A named financial threshold you define for cost, usage, RI/SP coverage	'Q3-EC2-Budget' set to $10,000/month
Budget Type	What the budget tracks: Cost, Usage, RI Utilisation, RI Coverage, SP Utilisation, SP Coverage	Cost budget on EC2; Usage budget on S3 GB stored
Budget Period	Time window: Daily, Monthly, Quarterly, Annually, or Custom date range	Monthly budget resets on 1st of each month
Actual vs Forecast	Alert on actual spend already incurred OR projected end-of-period spend	Alert at 85% actual OR 100% forecasted by month-end
Alert Threshold	Percentage or absolute dollar amount that triggers a notification	Alert at 50%, 80%, 100% of $10,000 = $5K, $8K, $10K
Budget Action	Automated response when a threshold is crossed	Deny IAM policy, stop EC2 instances, or run SSM document
Notification	Email or SNS topic triggered when threshold is reached	Email CFO + Slack channel when 80% reached
Filter	Scope budget to specific services, accounts, regions, tags, etc.	Budget only for tag Project=Phoenix in us-east-1

Term	Definition	Example
Cost Allocation Tag	User-defined key-value tags that appear as cost dimensions in budgets	Team=DataEngineering, Env=Production
Linked Accounts	In AWS Organisations, budget can span all or specific member accounts	Budget covering accounts 111, 222, 333 together
RI Utilisation	Percentage of purchased Reserved Instance hours actually used	Alert if RI utilisation drops below 80%
SP Coverage	Percentage of on-demand spend covered by Savings Plans	Alert if Savings Plans coverage drops below 70%

Budget Types at a Glance

Budget Type	Tracks	Alert Triggers On	Common Use Case
Cost Budget	Dollar spend	$ actual or $ forecasted	Monthly EC2 spend > $5,000
Usage Budget	Service usage units	Unit actual or forecasted	S3 storage > 100 TB; Lambda > 1M requests
RI Utilisation	% of RI hours used	Utilisation < threshold	Alert if RIs are under-used (wasted spend)
RI Coverage	% of usage covered by RIs	Coverage < threshold	Alert if too much On-Demand is used (should buy RIs)
SP Utilisation	% of SP commitment used	Utilisation < threshold	Alert if Savings Plans are under-utilised
SP Coverage	% of spend covered by SPs	Coverage < threshold	Alert if insufficient SP coverage

3. Architecture & Working



Step	What Happens
1. Define Budget	Set type, amount, period, and filters (service, region, tag, account)
2. Set Alert Thresholds	Define up to 5 thresholds per budget (e.g. 50%, 80%, 100% actual + 100% forecasted)
3. Configure Actions	Optionally attach automated actions: IAM policy, EC2/RDS stop, SSM document
4. Continuous Evaluation	Budgets engine compares actual/forecasted spend to thresholds up to 3x/day
5. Threshold Crossed	Email or SNS notification sent; Budget Action triggered (auto or with approval)
6. Monthly Reset	For monthly budgets, counters reset on the 1st of each month

4. Features

Feature	Description
Multiple Budget Types	Track cost, usage, RI utilisation/coverage, and Savings Plans — six distinct budget types
Forecast-Based Alerts	Alert not just on what you've spent but on what you're projected to spend by period end
Budget Actions	Automatically apply IAM deny policies, stop EC2/RDS instances, or run SSM runbooks when budget exceeded
Custom Filters	Scope to specific services, regions, AZs, accounts, tags, purchase type, or API operations
Multi-Account Support	Create organisation-level budgets spanning all linked accounts in AWS Organisations
Cost Allocation Tags	Filter budgets by custom tags — track spend by team, project, application, environment
Up to 5 Alert Thresholds	Define up to 5 graduated notifications per budget (e.g. 50%, 75%, 90%, 100%, 110%)

Feature	Description
SNS Integration	Publish to any SNS topic — connect to Slack, PagerDuty, Chime, email lists, Lambda
AWS Budgets Reports	Scheduled PDF/CSV reports of all your budgets delivered via email
Anomaly Detection Alerts	Integrates with AWS Cost Anomaly Detection for ML-based spend spike alerts
Granularity	Daily, monthly, quarterly, or annual periods — or custom start/end dates
Free Tier	First 2 budgets per month free; $0.02/budget/day thereafter

5. Real-World Use Cases

Industry / Team	Use Case
Startup	Monthly cost budget of $2,000 on all services — email CTO at 80%, auto-stop dev EC2 at 100%
Enterprise FinOps	Per-team budgets (tag Team=DataEng) with Slack alerts — democratise cost ownership
E-Commerce	Forecast-based alert: if projected month-end bill exceeds $50K, page on-call engineer
Healthcare	RI Coverage budget — alert if less than 75% of compute is covered by Reserved Instances (wasting money)
Consulting Firm	Per-client budgets using cost allocation tags (tag Client=AcmeCorp) — direct billing back
Dev/Test Governance	Budget action: automatically apply IAM Deny policy to dev accounts on the 25th of each month
Multi-Account Org	Organisation-wide monthly budget across 30 accounts — central FinOps team alerted
SaaS Provider	Per-environment budgets (Prod, Staging, Dev) — prevent staging from outspending production

6. Practical Example

Scenario: Create a monthly $10,000 EC2 cost budget with 3 alert thresholds and an automatic action to stop non-production instances when 100% is reached.

```
aws budgets create-budget \ --account-id 123456789012 \ --budget '{ "BudgetName":
"EC2-Monthly-10K", "BudgetType": "COST", "BudgetLimit": {"Amount": "10000", "Unit": "USD"},
"TimeUnit": "MONTHLY", "CostFilters": {"Service": ["Amazon Elastic Compute Cloud - Compute"]},
"CostTypes": {"IncludeTax": true, "IncludeSupport": false, "UseBlended": false, "UseAmortized":
false} }' \ --notifications-with-subscribers '[ { "Notification":
{"NotificationType":"ACTUAL","ComparisonOperator":"GREATER_THAN", "Threshold":
80,"ThresholdType":"PERCENTAGE"}, "Subscribers": [
{"SubscriptionType":"EMAIL","Address":"finops@company.com"},
{"SubscriptionType":"SNS","Address":"arn:aws:sns:us-east-1:123:budget-alerts"} ] }, {
"Notification": {"NotificationType":"ACTUAL","ComparisonOperator":"GREATER_THAN", "Threshold":
100,"ThresholdType":"PERCENTAGE"},
```

```
"Subscribers":[{"SubscriptionType":"EMAIL","Address":"cto@company.com"}] }, { "Notification":
{"NotificationType":"FORECASTED","ComparisonOperator":"GREATER_THAN", "Threshold":
100,"ThresholdType":"PERCENTAGE"},
"Subscribers":[{"SubscriptionType":"EMAIL","Address":"finops@company.com"}] } ]' aws budgets
create-budget-action \ --account-id 123456789012 \ --budget-name EC2-Monthly-10K \
--notification-type ACTUAL \ --action-type STOP_EC2_INSTANCES \ --action-threshold
'{"ActionThresholdValue":100,"ActionThresholdType":"PERCENTAGE"}' \ --definition
'{"ScpActionDefinition":{},
"Ec2ActionDefinition":{"InstanceIds":["i-dev1","i-dev2"],"Region":"us-east-1"}}' \
--execution-role-arn arn:aws:iam::123:role/BudgetActionsRole \ --approval-model AUTOMATIC \
--subscribers '[{"SubscriptionType":"EMAIL","Address":"finops@company.com"}]'
```

■ **Result:** At 80% ($8,000): email + SNS alert to FinOps team. At 100% ($10,000) actual: CTO emailed. At 100% forecasted: FinOps alerted early. At 100% actual: dev EC2 instances automatically stopped — preventing further spend.

7. Integrations

AWS Service	Integration
Amazon SNS	Publish budget breach notifications to SNS — fan out to email, Slack Lambda, PagerDuty
AWS Cost Explorer	Budget data visible in Cost Explorer; drill into cost drivers when alerted
AWS Cost Anomaly Detection	Integrates budget alerts with ML-based anomaly detection for unusual spend patterns
AWS Organizations	Manage budgets across all accounts; create org-level budgets in management account
Amazon EC2 / RDS	Budget Actions can stop EC2 instances or RDS clusters when budget exceeded
AWS IAM	Budget Actions apply IAM deny policies to users/roles/groups when budget breached
AWS Systems Manager	Budget Actions trigger SSM automation runbooks for custom cost-saving responses
AWS Lambda	Via SNS → Lambda — custom alerting logic, Slack/Teams notifications, ticket creation
AWS CloudWatch	Budget metrics can trigger CloudWatch alarms for additional monitoring coverage
Amazon Chime / Slack	Via SNS topic → Lambda → Slack/Chime webhook for team cost alerts in chat

8. Comparison

Feature	AWS Budgets	AWS Cost Explorer	AWS Cost Anomaly Detection	CloudWatch Billing Alarm
Purpose	Set thresholds + alert + act	Analyse historical/forecast cost	Detect unexpected spend spikes	Alert on total bill

Feature	AWS Budgets	AWS Cost Explorer	AWS Cost Anomaly Detection	CloudWatch Billing Alarm
Proactive/Reactive	■ Proactive (alert before)	Reactive (after the fact)	Both (ML-based)	Reactive
Granularity	Service, tag, account, region	Same (interactive exploration)	Service / account	Account total only
Automation	■ Budget Actions	■ No automation	■ No automation	Via CloudWatch action
Forecast-based	■ Yes	■ Yes (view only)	N/A (anomaly-based)	■ Actual only
RI/SP tracking	■ Dedicated budget types	■ View only	■ No	■ No
Cost	$0.02/budget/day (2 free)	Included (pay for advanced query)	Free (first $10/monitor free)	Free
Best for	Guardrails + automated control	Deep cost analysis + reporting	Catching runaway spend	Simple total bill alarm

■ AWS Cost Explorer

Visualise, Understand, and Forecast Your AWS Costs and Usage

1. Simple Introduction

AWS Cost Explorer is an **interactive data visualisation and cost analysis tool** that lets you explore, understand, and forecast your AWS costs and usage over time. It provides pre-built reports, custom filtering, grouping by any dimension (service, region, tag, account), and ML-powered forecasts — all through a browser console or API.

■ **Real-World Analogy:** AWS Cost Explorer is like a **personal finance app (like Mint or YNAB) for your AWS bill**. Just as Mint shows your spending by category (groceries, dining, travel) over time with trends and forecasts, Cost Explorer shows your AWS spending by service (EC2, S3, RDS), by team (via tags), or by region — with charts, tables, and projections up to 12 months into the future.

Business Problems Solved:

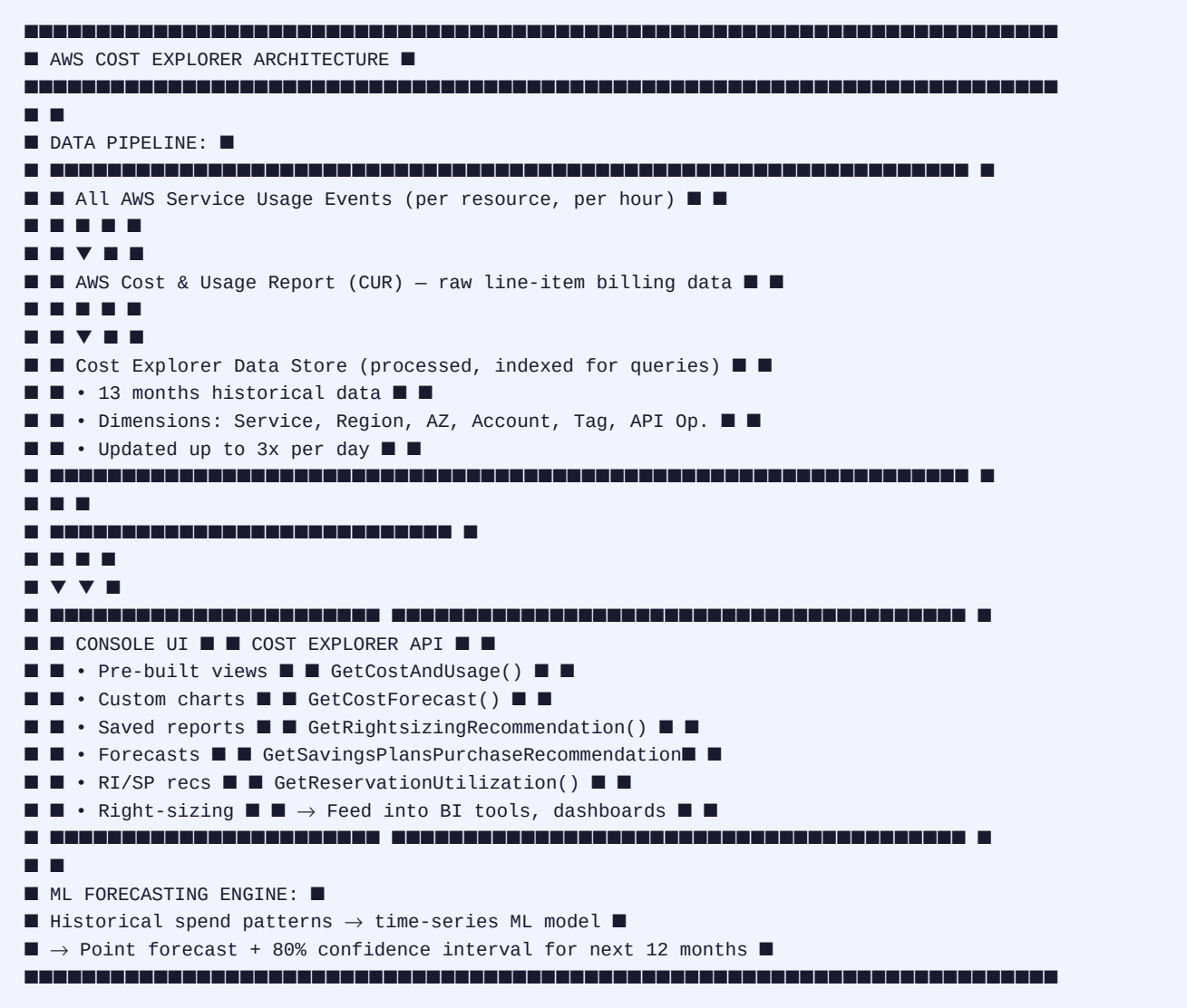
- Answer 'Where is all my AWS money going?' with interactive, filterable charts
- Identify cost trends and anomalies before the finance team asks about them
- Get right-sizing recommendations to eliminate wasted EC2 spend
- Forecast next month's bill to plan budgets and negotiate Reserved Instances
- Understand Savings Plans and Reserved Instance savings vs on-demand cost

2. Core Concepts

Term	Definition	Example
Cost & Usage Report (CUR)	The underlying raw billing dataset that powers Cost Explorer	Line-item detail: every API call, per hour, with cost
Time Range	The period you are analysing — up to 13 months historical	Last 6 months, last 3 months, or custom date range
Granularity	How data is grouped in time: Daily, Monthly, or Hourly (requires extra)	Monthly bars show cost per month; daily for trend analysis
Group By	Break down costs by a dimension or tag	Group by Service, Region, Account, Tag:Team
Filter	Narrow results to specific values of a dimension	Filter to Service=EC2 and Region=us-east-1 only
Dimension	A standard cost attribute you can group or filter by	Service, Region, Availability Zone, Account, API Operation
Tag	User-defined key-value that becomes a cost dimension after activation	Tag: Project=Phoenix — filter to see only Phoenix costs
Amortised Cost	Upfront RI/SP costs spread across the reservation period	3-year RI upfront fee divided over 36 months
Blended Rate	Average rate across all instances (used vs unused) in a family	Blended EC2 rate includes RI and On-Demand mixed

Term	Definition	Example
Unblended Cost	Actual individual line-item cost (most common view)	What you actually paid per hour per resource
Forecast	ML-predicted future spend based on historical patterns	Projected: $23,400 by end of month (with confidence interval)
Savings Plans Recs	Recommendations for Savings Plans commitment to maximise savings	Buy $5.00/hr Compute SP to save $1,200/month
RI Recommendations	Recommendations for Reserved Instance purchases based on usage	Buy 3x m5.xlarge 1-year RIs to save $450/month
Right-Sizing Recs	EC2 right-sizing suggestions based on CloudWatch utilisation metrics	Downsize i-abc from m5.2xlarge to m5.large — save $120/mo

3. Architecture & Working



Step	Action
1. Enable	Enable Cost Explorer in the AWS Console (takes 24 hours to populate initial data)

Step	Action
2. Activate Tags	Activate cost allocation tags in Billing console so they appear as filter dimensions
3. Open Console	Navigate to Cost Explorer — default view shows last 6 months by service
4. Filter & Group	Apply filters (service, region, tag) and group by (account, tag, AZ) to slice data
5. Forecast	Extend date range to future months — Cost Explorer shows ML forecast with confidence band
6. Save Reports	Save custom views as named reports for repeated access
7. API Access	Use GetCostAndUsage API to pull data into custom dashboards, QuickSight, or spreadsheets
8. Act on Recs	Review RI/SP/right-sizing recommendations and purchase/resize accordingly

4. Features

Feature	Description
Interactive Charts	Bar, line, and stacked charts of cost/usage — zoom, filter, group interactively
13 Months History	Access up to 13 months of historical cost and usage data
12-Month Forecast	ML-based forecast of future spend with confidence interval for planning
Cost Allocation Tags	Filter and group by custom tags (team, project, env) after activation in Billing
Savings Plans Recommendations	Hourly Compute or EC2 Savings Plans purchase recommendations with projected savings
RI Purchase Recommendations	Recommended RI purchases per instance type, term, and payment option
EC2 Right-Sizing	Identify oversized EC2 instances using CloudWatch CPU/memory data with cost impact
Cost Anomaly Detection	ML detects unusual spending patterns and alerts via SNS/email
Saved Reports	Save any filtered/grouped view as a named report for repeatable access
Cost Explorer API	Programmatic access to cost data — build custom dashboards, scripts, automation
Granular Data	Filter by API operation — see cost per specific AWS API call
Linked Account View	Management account sees all member account costs in one consolidated view
Amortised vs Blended vs Net	Multiple cost calculation methods to suit finance team preferences
CSV Export	Export any report to CSV for Excel/BI tools

5. Real-World Use Cases

Industry / Role	Use Case
FinOps Team	Monthly cost review: identify which team's S3 spend grew 40% this month — drill into tag=Team:DataEng
Engineering Manager	Right-sizing: Cost Explorer flags 15 dev EC2 instances at < 5% CPU — downsize to save $3,000/month
CFO / Finance	12-month forecast: project next quarter's AWS spend for board presentation and budget planning
Cloud Architect	Compare on-demand vs RI vs Savings Plans costs — quantify the savings from purchasing SPs last quarter
Startup CTO	Weekly cost review by service — monitor if S3 data transfer or Lambda invocations are growing unexpectedly
Enterprise FinOps	Per-account cost breakdown across 50 AWS accounts — identify top 5 spenders for optimization focus
DevOps	API integration: pull daily cost data into internal Grafana dashboard for engineering team visibility
Procurement	RI recommendations: buy recommended m5.xlarge RIs to save $12,000/year vs current on-demand usage

6. Practical Example

Scenario: Use the Cost Explorer API to retrieve last month's top 5 services by cost and get an end-of-month forecast.

```
import boto3 from datetime import date, timedelta ce = boto3.client('ce', region_name='us-east-1') today = date.today() first_of_month = today.replace(day=1) last_month_start = (first_of_month - timedelta(days=1)).replace(day=1) last_month_end = first_of_month response = ce.get_cost_and_usage( TimePeriod={ 'Start': last_month_start.strftime('%Y-%m-%d'), 'End': last_month_end.strftime('%Y-%m-%d'), }, Granularity='MONTHLY', Metrics=['UnblendedCost'], GroupBy=[{'Type': 'DIMENSION', 'Key': 'SERVICE'}], Filter={'Not': {'Dimensions': {'Key':'RECORD_TYPE','Values':['Credit']}}} ) results = response['ResultsByTime'][0]['Groups'] sorted_svcs = sorted(results, key=lambda x: float(x['Metrics']['UnblendedCost']['Amount']), reverse=True) print('Top 5 Services Last Month:') for i, svc in enumerate(sorted_svcs[:5], 1): name = svc['Keys'][0] cost = float(svc['Metrics']['UnblendedCost']['Amount']) print(f' {i}. {name}: ${cost:,.2f}') end_of_month = today.replace(day=28) + timedelta(days=4) end_of_month = end_of_month - timedelta(days=end_of_month.day - 1) end_of_month = end_of_month.replace(month=end_of_month.month % 12 + 1 if end_of_month.month < 12 else 1) forecast = ce.get_cost_forecast( TimePeriod={'Start': today.strftime('%Y-%m-%d'), 'End': end_of_month.strftime('%Y-%m-%d')}, Metric='UNBLENDED_COST', Granularity='MONTHLY', PredictionIntervalLevel=80 ) fcast_amt = float(forecast['Total']['Amount']) lower = float(forecast['ForecastResultsByTime'][0]['PredictionIntervalLowerBound']) upper = float(forecast['ForecastResultsByTime'][0]['PredictionIntervalUpperBound']) print(f'\nForecast for this month: ${fcast_amt:,.2f} (80% CI: ${lower:,.0f} – ${upper:,.0f})')
```

■ **Result:** Script prints ranked services by cost last month and the ML-powered forecast for the current month with an 80% confidence interval — ready to pipe into dashboards, Slack bots, or budget planning spreadsheets.

7. Integrations

AWS Service	Integration
AWS Budgets	Budgets consume Cost Explorer data for threshold evaluation; CE shows budget trends
AWS Cost & Usage Report	CUR is the raw data source that populates all Cost Explorer views and APIs
AWS Cost Anomaly Detection	CE surfaces anomaly detection alerts; shares the same underlying cost data
Amazon QuickSight	Pull Cost Explorer API data into QuickSight for custom BI dashboards
Amazon S3	Export CUR (raw data) to S3 for custom analysis with Athena or Redshift Spectrum
Amazon Athena	Query CUR data exported to S3 with SQL for custom cost analysis beyond CE UI
AWS Organizations	Management account sees consolidated cost data across all member accounts in CE
AWS Trusted Advisor	TA cost optimisation checks complement CE right-sizing recommendations
AWS Compute Optimizer	CO provides deeper right-sizing analysis; CE shows the cost impact of recommendations
Amazon CloudWatch	CE right-sizing uses CloudWatch EC2 CPU/memory utilisation metrics

8. Comparison

Feature	Cost Explorer	AWS Budgets	Cost & Usage Report	AWS Compute Optimizer
Purpose	Visualise & analyse cost	Set alerts & guardrails	Raw billing data	Right-size compute
Proactive	■ Reactive (historical+forecast)	■ Yes (alert before)	■ Data only	■ Recommendations
Automation	■ No automated action	■ Budget Actions	■ No	■ No (manual action)
Granularity	Service/tag/region/account	Same filters	Line-item per resource/hour	Per EC2/ECS/Lambda
API Access	■ Cost Explorer API	■ Budgets API	S3 + Athena/Redshift	■ API
ML Forecast	■ 12-month forecast	■ Forecasted alerts	■ Raw data only	■ No
Cost	Free (advanced queries: $0.01)	$0.02/budget/day (2 free)	Free report + S3 storage	Free (or EC2 only)
Best for	Understanding & forecasting	Guardrails & alerts	Custom BI/deep analysis	Instance optimisation

■ Amazon API Gateway

Create, Publish, Maintain, Monitor, and Secure APIs at Any Scale

1. Simple Introduction

Amazon API Gateway is a **fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale**. It acts as the front door for applications to access backend services — Lambda functions, EC2, ECS, DynamoDB, or any HTTP endpoint — handling all the complexity of request routing, authentication, rate limiting, caching, and monitoring.

■ **Real-World Analogy:** API Gateway is like the **front desk receptionist at a large company**. Every visitor (API request) goes through the front desk. The receptionist checks ID (authentication), checks if the visitor has an appointment (authorisation), logs the visit (access logging), directs them to the right office (routing to backend), and tracks how many visitors arrived (metrics). Visitors never talk directly to the backend — everything flows through the front desk.

Business Problems Solved:

- Build production-ready REST, HTTP, or WebSocket APIs without managing server infrastructure
- Protect backends from traffic spikes with built-in throttling and rate limiting
- Authenticate API consumers with IAM, Cognito, Lambda authorisers, or API keys
- Reduce backend load and latency with built-in response caching
- Monitor every API call with CloudWatch metrics, logs, and X-Ray distributed tracing

2. Core Concepts

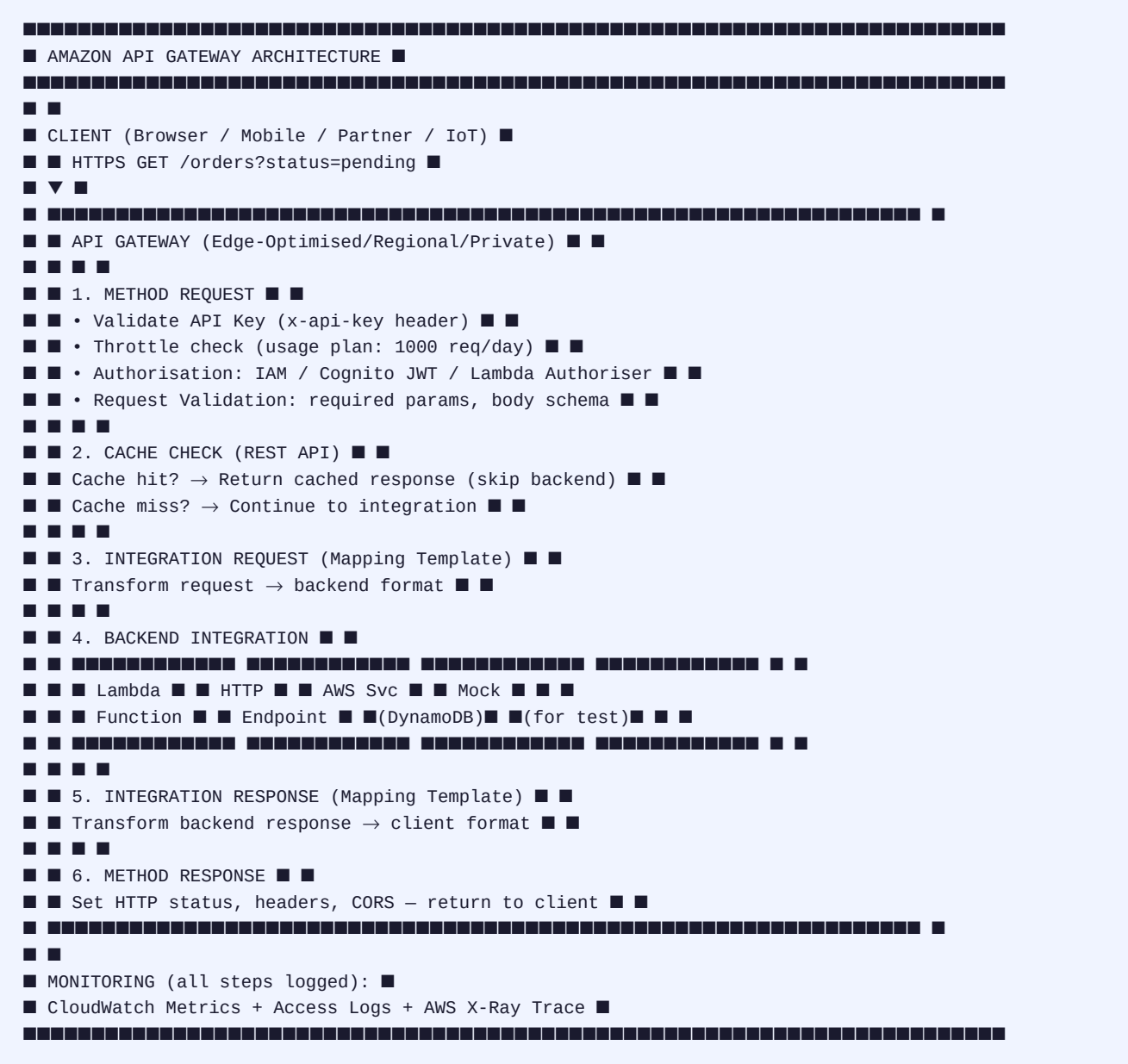
Term	Definition	Example
API	The top-level resource — a collection of routes and stages	my-orders-api (REST API)
API Type	REST API, HTTP API, or WebSocket API — each with different features/pricing	HTTP API for simple proxy; REST API for full features
Resource	A path component in a REST API — represents a noun	/orders, /orders/{orderId}, /products
Method	An HTTP verb on a resource — GET, POST, PUT, DELETE, PATCH	GET /orders — list all orders
Integration	The backend that handles the request — Lambda, HTTP, AWS service, Mock	POST /orders → Lambda function create-order
Stage	A named deployment snapshot of the API	dev, staging, prod — each has its own URL and settings
Deployment	Pushing API configuration to a stage	Deploy v2 of the API to the prod stage
Route	HTTP API term — a method + path combination	POST /orders, GET /orders/{id}

Term	Definition	Example
Authoriser	Component that validates the identity/permissions of the caller	JWT authoriser (Cognito), Lambda authoriser (custom logic)
Usage Plan	Throttle and quota settings linked to an API key	Free tier: 1,000 requests/day, 10 req/sec; Premium: 100K/day
API Key	A string token identifying a calling application for usage tracking	abc123xyz embedded in x-api-key header
Throttling	Rate limits: burst (peak TPS) and steady-state (avg TPS) per stage/route	10,000 burst / 5,000 RPS steady-state
Caching	Cache GET responses for a TTL — return cached response without hitting backend	Cache /products list for 300 seconds
Mapping Template	VTL (Velocity Template Language) to transform request/response bodies	Transform flat JSON to DynamoDB's nested request format
CORS	Cross-Origin Resource Sharing — allows browser apps from other domains to call the API	Allow https://myapp.com to call the API from a browser
WebSocket API	Full-duplex persistent connection for real-time two-way communication	Live chat, stock ticker, multiplayer game events
Private API	REST API accessible only within a VPC via a VPC endpoint	Internal microservices API not exposed to internet
Canary Deployment	Send a % of prod traffic to a new API version for safe rollout	Send 5% of traffic to v2 canary; if stable, shift 100%

API Types Comparison

Feature	REST API	HTTP API	WebSocket API
Protocol	HTTP/HTTPS	HTTP/HTTPS	WebSocket (ws/wss)
Communication	Request-response	Request-response	Full-duplex persistent
Pricing	~$3.50/million requests	~$1.00/million (71% cheaper)	$1.00/million messages
Caching	■ Yes (built-in)	■ No	■ No
Usage Plans/Keys	■ Yes	■ No	■ No
Request Validation	■ Yes	■ No	■ No
Mapping Templates	■ Yes (VTL)	■ No	■ No
JWT Auth (native)	■ Via Lambda authoriser	■ Built-in JWT auth	■ No
Private API	■ Yes	■ Yes	■ No
AWS X-Ray	■ Yes	■ Yes	■ Limited
Best for	Full-featured enterprise API	Low-latency Lambda/HTTP proxy	Real-time bidirectional

3. Architecture & Working



4. Features

Feature	Description
Multiple Auth Methods	IAM SigV4, Cognito User Pool JWT, Lambda custom authoriser, API key — mix per route
Request Validation	Validate required query params, headers, and body schema before hitting backend — reject bad requests early
Response Caching	Cache backend responses for a TTL (1 sec – 1 hour) — reduce latency and backend cost (REST API)

Feature	Description
Throttling & Quotas	Stage-level and route-level burst + steady-state limits; per-key quotas via usage plans
Custom Domain Names	Use api.mycompany.com instead of execute-api.us-east-1.amazonaws.com via Route 53 + ACM
Canary Deployments	Route a percentage of traffic to a new stage version for gradual rollout
Mutual TLS (mTLS)	Require clients to present X.509 certificates for machine-to-machine API security
VPC Link	Connect API Gateway to private resources in a VPC (ALB, NLB, ECS) without public internet
WebSocket Support	Manage persistent connections; route messages to Lambda by message content
X-Ray Tracing	End-to-end distributed tracing from API Gateway through Lambda to downstream services
CloudWatch Access Logs	Log every request with caller IP, latency, status code, error message — full audit trail
Mock Integration	Return static responses without a backend — for API-first development and testing
SDK Generation	Auto-generate client SDKs for JavaScript, iOS, Android, Java from REST API definition
OpenAPI / Swagger Import	Import existing OpenAPI 3.0 / Swagger 2.0 definitions to create APIs instantly
WAF Integration	Attach AWS WAF to protect API from SQL injection, XSS, rate-based attacks

5. Real-World Use Cases

Industry	Use Case
E-Commerce	REST API: /products, /cart, /orders backed by Lambda + DynamoDB — serverless e-commerce backend
Banking	Secure API with mTLS + Lambda authoriser for fintech partner integrations — zero server management
Healthcare	FHIR R4 API Gateway exposing patient data to authorised healthcare apps — Cognito JWT auth per practitioner
Startup	HTTP API in front of Lambda — 71% cheaper than REST, perfect for simple CRUD microservices
IoT	Devices POST telemetry to API Gateway → Lambda → DynamoDB/Kinesis — fully serverless ingestion
Gaming	WebSocket API for real-time multiplayer game events — persistent bi-directional player connections

Industry	Use Case
Enterprise	Private REST API inside VPC for internal microservices — VPC Link to ECS/ALB backends
SaaS	Usage Plans + API Keys to tier customers (Free 1K/day, Pro 100K/day) — built-in monetisation
Media	Content delivery API: API Gateway + CloudFront + S3 — CDN-cached media metadata API
DevOps	Mock integration during development — return static responses while backend teams build services

6. Practical Example

Scenario: Create a serverless REST API with two endpoints: GET /orders (list orders from DynamoDB) and POST /orders (create order via Lambda), with Cognito JWT authentication and custom domain.

```
aws apigateway create-rest-api \ --name orders-api \ --endpoint-configuration types=REGIONAL \
--description 'Orders microservice REST API' API_ID=$(aws apigateway get-rest-apis \ --query
'items[?name==`orders-api`].id' --output text) ROOT_ID=$(aws apigateway get-resources
--rest-api-id $API_ID \ --query 'items[?path==`/`].id' --output text) RESOURCE_ID=$(aws apigateway
create-resource \ --rest-api-id $API_ID \ --parent-id $ROOT_ID \ --path-part orders \ --query 'id'
--output text) aws apigateway create-authorizer \ --rest-api-id $API_ID \ --name cognito-auth \
--type COGNITO_USER_POOLS \ --provider-ar-ns
arn:aws:cognito-idp:us-east-1:123:userpool/us-east-1_ABC \ --identity-source
'method.request.header.Authorization' aws apigateway put-method \ --rest-api-id $API_ID \
--resource-id $RESOURCE_ID \ --http-method GET \ --authorization-type COGNITO_USER_POOLS \
--authorizer-id $AUTHORIZER_ID aws apigateway put-integration \ --rest-api-id $API_ID \
--resource-id $RESOURCE_ID \ --http-method GET \ --type AWS_PROXY \ --integration-http-method POST
\ --uri 'arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:12
3:function:list-orders/invocations' aws apigateway create-deployment \ --rest-api-id $API_ID \
--stage-name prod \ --stage-description 'Production' aws apigatewayv2 create-domain-name \
--domain-name api.mycompany.com \ --domain-name-configurations \
'CertificateArn=arn:aws:acm:us-east-1:123:certificate/abc,EndpointType=REGIONAL'
```

■ **Result:** REST API deployed at https://api.mycompany.com/prod/orders. GET /orders requires a valid Cognito JWT. Calls are proxied to the list-orders Lambda. All requests logged to CloudWatch with latency and status code.

7. Integrations

AWS Service	Integration
AWS Lambda	Most common integration — Lambda proxy handles request and returns response (serverless backend)
Amazon Cognito	Cognito User Pool JWT authoriser — authenticate users without writing auth code
Amazon DynamoDB	Direct AWS service integration — API Gateway calls DynamoDB API without Lambda
Amazon S3	Proxy GET requests to S3 objects — serve files or pre-signed URLs via API
AWS WAF	Attach WAF Web ACL to API Gateway stage for DDoS and injection protection

AWS Service	Integration
Amazon CloudFront	Put CloudFront in front of API Gateway for edge caching and global distribution
AWS X-Ray	Enable active tracing — visualise full request path from API Gateway to Lambda to DB
Amazon CloudWatch	Detailed metrics (latency, 4xx/5xx errors, count) and access log streams per stage
Amazon Kinesis	Direct integration — stream data to Kinesis Data Streams without Lambda
AWS Step Functions	Trigger state machine executions directly from API Gateway
Amazon SQS	Enqueue messages to SQS directly — decoupled async API ingestion pattern
AWS Certificate Manager	TLS certificates for custom domain names

8. Comparison

Feature	API Gateway REST	API Gateway HTTP	API Gateway WebSocket	ALB (HTTP)	AWS App
Protocol	HTTP/HTTPS	HTTP/HTTPS	WebSocket	HTTP/HTTPS	GraphQL
Real-time	■ Request/Reply	■ Request/Reply	■ Persistent	■ No	■ Subscrip
Caching	■ Built-in	■ No	■ No	■ No	■ Yes
Auth	IAM/Cognito/Lambda	JWT/Lambda/IAM	Lambda authoriser	Cognito/OIDC	Cognito/IAM DC
Usage Plans	■ Yes	■ No	■ No	■ No	■ No
Transformation	■ VTL templates	■ No	■ No	■ No	■ Resolver
Price/M reqs	~$3.50	~$1.00	~$1.00/M msgs	~$0.008/LCU-hr	~$4.00
Best for	Enterprise REST API	Simple Lambda proxy	Real-time chat/games	Containerised services	GraphQL / real-time da

■ Amazon SageMaker

Build, Train, and Deploy Machine Learning Models at Scale — End-to-End ML Platform

1. Simple Introduction

Amazon SageMaker is a **fully managed, end-to-end machine learning platform** that provides every tool a data scientist or ML engineer needs — from data preparation and model training to deployment, monitoring, and MLOps automation — all in one integrated service. SageMaker removes the undifferentiated heavy lifting of infrastructure management so teams can focus on building better models.

■ **Real-World Analogy:** Amazon SageMaker is like a **fully equipped smart factory for building AI products**. The factory has every machine you need: raw material intake (data preparation), assembly lines (training), quality control (model evaluation), packaging (deployment), and a continuous production monitor (model monitoring). Workers (data scientists) walk in, design the product, and the factory handles all the machinery, electricity, and supply chain — they just focus on what to build.

Business Problems Solved:

- Eliminate infrastructure management for ML — no GPU cluster setup, patching, or scaling
- Accelerate model development with managed Jupyter notebooks and built-in algorithms
- Train on large datasets efficiently using distributed training across GPU clusters
- Deploy models as scalable, low-latency REST endpoints with one API call
- Monitor production models for data drift and degradation automatically

2. Core Concepts

Term	Definition	Example
SageMaker Studio	Web-based integrated development environment (IDE) for the complete ML lifecycle	Jupyter-like IDE with model registry, pipelines, experiments all integrated
Notebook Instance	Managed Jupyter notebook server on EC2 — pre-configured for ML	ml.t3.medium notebook for exploration; ml.p3.2xlarge for training notebooks
Training Job	A managed compute job that runs your training script on specified instances	Train XGBoost on 10M rows using 4x ml.m5.xlarge instances
Training Script	Your Python code that defines the model architecture and training loop	train.py with TensorFlow/PyTorch/Scikit-learn code
Built-in Algorithm	Pre-built, optimised ML algorithms ready to use without writing training code	XGBoost, Linear Learner, DeepAR, BlazingText, Image Classification
Model	The trained artefact — stored in S3 as model.tar.gz	model.tar.gz containing weights, architecture, preprocessing pipeline

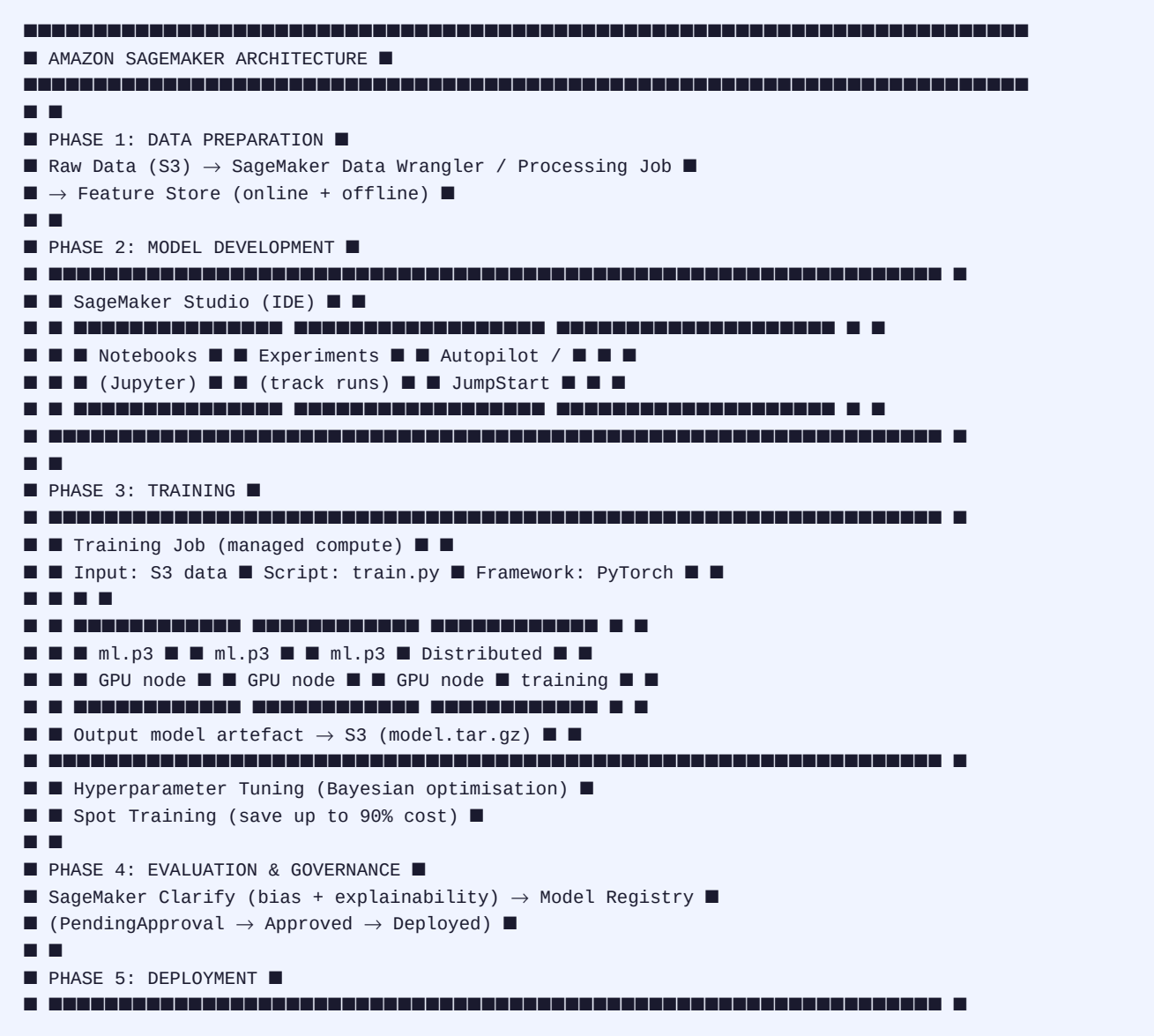
Term	Definition	Example
Endpoint	A managed REST API serving real-time predictions from a deployed model	https://runtime.sagemaker.us-east-1.amazonaws.com/endpoints/my-model/invocations
Endpoint Config	Specifies which model(s) and instance types to use for an endpoint	Production variant: ml.m5.large × 2; Canary: ml.m5.large × 1
Batch Transform	Run inference on large datasets without a persistent endpoint	Score 1M rows overnight; results saved to S3 — no endpoint needed
SageMaker Pipelines	CI/CD for ML — automate the full workflow from data prep to deployment	10-step pipeline: preprocess → train → evaluate → register → deploy
Model Registry	Centralised catalogue of trained model versions with approval workflow	v1 Approved, v2 PendingApproval, v3 Rejected — governance layer
Experiments	Track and compare training runs — hyperparameters, metrics, artefacts	Compare 20 runs varying learning rate and batch size — pick best
Feature Store	Centralised managed repository for ML features — online (low-latency) and offline (batch)	customer_age, purchase_history features shared across 5 models
SageMaker Clarify	Explain model predictions and detect bias in data and models	SHAP values per feature; detect if model is biased against age group
SageMaker Monitor	Monitor live endpoints for data quality, model quality, bias, and feature drift	Alert when % of null values in input exceeds 5%
Autopilot	AutoML — automatically build, train, and tune the best model for tabular data	Upload CSV, let Autopilot try 250 model configurations, return the best
JumpStart	One-click deployment of pre-trained foundation models and ML solutions	Deploy Llama 3, Stable Diffusion, or Hugging Face BERT in one click
Hyperparameter Tuning	Automated search for optimal hyperparameters using Bayesian optimisation	Search across learning_rate, max_depth, n_estimators to minimise validation loss
Spot Training	Use EC2 Spot Instances for training to save up to 90% of training cost	ml.p3.8xlarge Spot: $3.06/hr vs $12.24/hr On-Demand
SageMaker Canvas	No-code ML tool — business analysts build models without writing code	Upload sales data CSV → Canvas builds a sales forecast model

SageMaker Instance Types

Family	Purpose	Examples	Best For
ml.t3 / ml.m5	General purpose	ml.t3.medium, ml.m5.xlarge	Notebooks, light training, small endpoints
ml.c5	Compute optimised	ml.c5.4xlarge, ml.c5.9xlarge	CPU-bound training, inference

Family	Purpose	Examples	Best For
ml.p3	GPU (V100)	ml.p3.2xlarge, ml.p3.16xlarge	Deep learning training — NLP, CV
ml.p4d	GPU (A100)	ml.p4d.24xlarge	Large model training, multi-GPU
ml.g4dn	GPU inference (T4)	ml.g4dn.xlarge, ml.g4dn.12xl	Cost-effective GPU inference
ml.g5	GPU (A10G)	ml.g5.xlarge, ml.g5.48xlarge	LLM inference, gen AI
ml.inf2	AWS Inferentia2	ml.inf2.xlarge, ml.inf2.48xl	Highest throughput/$ inference for Transformers
ml.trn1	AWS Trainium	ml.trn1.32xlarge	Cheapest large model training on AWS
ml.r5	Memory optimised	ml.r5.2xlarge, ml.r5.24xlarge	Large feature sets, in-memory processing

3. Architecture & Working



```
■ ■ Real-time Endpoint ■ Batch Transform ■ Async Endpoint ■ ■
■ ■ (low-latency) ■ (large datasets) ■ (large inputs) ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■
■ PHASE 6: MONITORING ■
■ SageMaker Monitor → detect data drift → alert → retrain trigger ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Phase	SageMaker Tool	Output
Data Prep	Data Wrangler, Processing Job, Feature Store	Clean features in S3 / Feature Store
Exploration	Studio Notebooks, Autopilot, JumpStart	Trained model prototype, best algorithm selected
Training	Training Jobs, Hyperparameter Tuning, Spot	model.tar.gz in S3
Evaluation	SageMaker Clarify, Experiments	Bias report, SHAP values, metrics comparison
Governance	Model Registry, Lineage Tracking	Approved model version ready for deployment
Deployment	Endpoints, Batch Transform, Async Endpoints	REST API or S3 result files
MLOps	Pipelines, Model Monitor, CloudWatch	Automated retrain triggers on drift detected

4. Features

Feature	Description
SageMaker Studio	Unified web IDE — notebooks, experiments, pipelines, registry, monitoring in one place
Managed Spot Training	Use Spot Instances for training — up to 90% savings; automatic checkpointing for interruption recovery
Built-in Algorithms	18+ optimised algorithms (XGBoost, DeepAR, BlazingText, K-NN, NTM) — no training code needed
Framework Containers	Managed Docker containers for TensorFlow, PyTorch, MXNet, Scikit-learn, Hugging Face
SageMaker Pipelines	Automated ML CI/CD pipelines with conditional steps, parameter overrides, caching
Model Registry	Versioned model catalogue with approval workflow — track lineage, deploy specific versions
Feature Store	Centralised feature repository — online (ms latency) and offline (S3) stores
SageMaker Clarify	Bias detection in data and models; SHAP-based explainability for any model
Model Monitor	Schedule monitoring jobs to detect data quality, model quality, and feature drift

Feature	Description
Autopilot	AutoML for tabular data — automatically tries algorithms and tunes hyperparameters
SageMaker JumpStart	Foundation model hub — deploy Llama, Mistral, Stable Diffusion, Falcon in one click
SageMaker Canvas	No-code ML for business users — point-and-click model building on tabular/image data
Distributed Training	SageMaker Data Parallel and Model Parallel libraries for large multi-GPU training
Multi-Model Endpoints	Host thousands of models on a single endpoint — cost-efficient inference at scale
Serverless Inference	Pay-per-invocation inference — no instance to keep warm; scales to zero
Real-time Inference	Persistent endpoint with auto-scaling; p99 latency monitoring; shadow variants
Batch Transform	Score large offline datasets without persistent endpoints — results to S3
Asynchronous Inference	Long-running inference jobs (large inputs, minutes runtime) with SNS completion notification
SageMaker HyperPod	Purpose-built cluster for training large foundation models with automatic recovery

5. Real-World Use Cases

Industry	Use Case	SageMaker Features Used
E-Commerce	Product recommendation engine	Training Jobs, Feature Store, Real-time Endpoint, Model Monitor
Banking	Credit risk scoring for loan applications	Autopilot (tabular), Clarify (bias), Batch Transform
Healthcare	Medical image classification (X-ray, MRI)	Training (PyTorch), GPU endpoints (g5), Clarify (explainability)
Retail	Demand forecasting for 100K SKUs	DeepAR built-in algorithm, Pipelines, Batch Transform
Manufacturing	Predictive maintenance on sensor data	Random Cut Forest anomaly detection, Streaming inference
Media	Content moderation — flag inappropriate images	Image Classification, Multi-Model Endpoint
Insurance	Claims fraud detection at submission time	Feature Store, XGBoost, Real-time low-latency endpoint
GenAI / LLM	Deploy and fine-tune Llama 3 / Mistral	JumpStart, Trainium/Inferentia instances, HyperPod

Industry	Use Case	SageMaker Features Used
Startup	Sales forecast from CRM data	Canvas (no-code) — business user builds model without data scientist
DevOps / MLOps	Automated model retraining pipeline	Pipelines, Model Registry, EventBridge trigger on drift alert

6. Practical Example

Scenario: Train an XGBoost model on tabular data in S3, register it, and deploy it as a real-time endpoint — all using the SageMaker Python SDK.

```
import sagemaker from sagemaker.xgboost import XGBoost from sagemaker import get_execution_role role = get_execution_role() sess = sagemaker.Session() bucket = sess.default_bucket() xgb = XGBoost( entry_point='train.py', framework_version='1.7-1', instance_type='ml.m5.xlarge', instance_count=1, role=role, use_spot_instances=True, max_wait=7200, max_run=3600, hyperparameters={ 'max_depth': 6, 'eta': 0.2, 'objective': 'binary:logistic', 'num_round': 100, 'eval_metric': 'auc', }, output_path=f's3://{bucket}/models/' ) xgb.fit({ 'train': f's3://{bucket}/data/train', 'validation': f's3://{bucket}/data/validation' }) model_package = xgb.register( model_package_group_name='churn-prediction-models', approval_status='PendingManualApproval', description='XGBoost churn model v2 — AUC 0.923' ) predictor = xgb.deploy( initial_instance_count=2, instance_type='ml.m5.large', endpoint_name='churn-prediction-endpoint' ) result = predictor.predict([[34, 1, 0.45, 8, 1200.50]]) print(f'Churn probability: {result}')
```

■ **Result:** XGBoost model trained on Spot Instances (saves ~70%). Model registered in Model Registry with PendingManualApproval status. After approval, deployed as a real-time endpoint on 2x ml.m5.large instances. Predictions served in < 10ms latency.

7. Integrations

AWS Service	Integration
Amazon S3	Training data input, model artefact output, batch transform results — S3 is SageMaker's data backbone
Amazon ECR	Store custom Docker training and inference container images
Amazon API Gateway	Expose SageMaker endpoints via API Gateway for external access with auth/throttling
AWS Lambda	Trigger training jobs, invoke endpoints, post-process predictions in event-driven pipelines
Amazon EventBridge	Model Monitor alerts trigger EventBridge → Lambda for automated retraining
AWS Step Functions	Orchestrate ML workflows — alternative to SageMaker Pipelines for multi-service workflows
Amazon CloudWatch	Training job metrics, endpoint invocation metrics, model quality metrics — full observability
AWS Glue	Preprocess and transform data in Glue; store in S3; SageMaker reads for training
Amazon Kinesis	Real-time streaming inference — Kinesis → Lambda → SageMaker endpoint

AWS Service	Integration
Amazon Redshift	Export data from Redshift to S3 via UNLOAD; SageMaker trains on the exported data
AWS KMS	Encrypt training data, model artefacts, and endpoints at rest
Amazon VPC	Run training jobs and endpoints in private VPC subnets — no public internet
Amazon Bedrock	SageMaker and Bedrock complement each other — Bedrock for foundation models, SageMaker for custom training
AWS IAM	Execution roles for training jobs, endpoints, pipelines — least-privilege ML access control

8. Comparison

Feature	SageMaker	Amazon Bedrock	Google Vertex AI	Azure ML	Self-Managed (EC2+K8s)
Type	End-to-end ML platform	Foundation model API	End-to-end ML platform	End-to-end ML platform	DIY ML infrastructure
Custom Training	■ Full support	■ No custom training	■ Yes	■ Yes	■ Yes (manual)
Foundation Models	■ JumpStart	■ Native (Claude, Titan)	■ Model Garden	■ Azure OpenAI	Manual setup
No-code ML	■ Canvas / Autopilot	■ Playground	■ AutoML	■ AutoML	■ No
MLOps / Pipelines	■ SageMaker Pipelines	■ No	■ Vertex Pipelines	■ Azure ML Pipelines	Kubeflow (complex)
Model Monitoring	■ Built-in	■ No	■ Yes	■ Yes	Custom Prometheus
Cost Savings	Spot training, Inf2, Trn1	Pay per token	Preemptible VMs	Spot VMs	Spot EC2 (manual)
AWS Native	■ Deep integration	■ Deep integration	■ GCP	■ Azure	■ EC2/ECR/S3
Best for	Custom model training/deploy	Consume LLMs via API	GCP ecosystem ML	Azure ecosystem ML	Maximum control

Quick-Reference Summary

Category	Service	Type	Key Strength	Be
■ FinOps	AWS Budgets	Cost Guardrails	Proactive alerts + automated actions	Prevent bill shock; e
■ FinOps	Cost Explorer	Cost Analytics	Interactive charts + ML forecast	Understand where n planning
■ Frontend	Amazon API Gateway	API Management	Managed auth, throttle, cache, logging	Serverless REST/HT
■ ML	Amazon SageMaker	ML Platform	End-to-end ML: prep → train → deploy → monitor	Custom model traini

Decision Flow

```
CLOUD COST MANAGEMENT: API / FRONTEND:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Need proactive guardrails? ■ ■ Building an HTTP API? ■
■ → AWS Budgets ■ ■ → API Gateway HTTP API ■
■ (alerts + automated actions) ■ ■ (simple, cheap, fast) ■
■ ■ ■ ■
■ Need to analyse & understand ■ ■ Need caching / usage plans ■
■ cost trends and forecasts? ■ ■ → API Gateway REST API ■
■ → AWS Cost Explorer ■ ■ (full enterprise features)■
■ (interactive BI + API) ■ ■ ■
■ ■ ■ Need real-time WebSocket? ■
■ Use BOTH together: ■ ■ → API Gateway WebSocket ■
■ Budgets = guardrails ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Cost Explorer = insights ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

MACHINE LEARNING:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Need to use a foundation model (LLM) without training? ■
■ → Amazon Bedrock (Claude, Titan, Llama via API) ■
■ ■
■ Need to train a CUSTOM model on your own data? ■
■ → Amazon SageMaker (full ML lifecycle) ■
■ No-code? → SageMaker Canvas / Autopilot ■
■ Code? → SageMaker Training Jobs + Studio ■
■ Real-time inference? → SageMaker Endpoints ■
■ Batch scoring? → SageMaker Batch Transform ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```